Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет
по лабораторной работе № 2
«Работа с коллекциями и встроенными структурами данных»

Выполнил: Писарик М. И.
студент группы 472301
Проверил: Мацокин Н. П.

Минск 2026

Цель: познакомиться с основными коллекциями и встроенными структурами данных в PHP, а также научиться эффективно применять функции обработки массивов и возможности SPL-классов.

Задачи:

– изучить синтаксис и семантику индексных, ассоциативных и многомерных массивов;

– освоить встроенные функции для работы с массивами: array_map, array_filter, array_reduce, сортировочные функции и операции над множествами;

– разобрать и применить SPL-структуры данных: SplStack, SplQueue, SplHeap, SplFixedArray, SplObjectStorage;

– научиться создавать собственные итераторы на основе интерфейсов Iterator и FilterIterator;

– реализовать на практике алгоритмы обработки коллекций (поиск, фильтрация, группировка);

– закрепить умение документировать контракты коллекций с помощью phpDoc и строгой типизации.

Теоретическая часть.

Массив в PHP объединяет динамический список и хеш-таблицу: он может быть индексным (целочисленные ключи), ассоциативным (строковые ключи) и многомерным (вложенные массивы). Элементы добавляются с помощью $arr[] = value, удаляются через array_pop/array_shift, а проверка наличия ключа выполняется функцией array_key_exists.

Для обработки коллекций в функциональном стиле используются функции высшего порядка:

– array_map(fn($x) => $x * $x, $nums) – преобразование каждого элемента;

– array_filter($nums, fn($x) => $x % 2 === 0) – отбор элементов по предикату;

– array_reduce($nums, fn($acc, $x) => $acc + $x, 0) – агрегация в одно значение.

Сортировка выполняется функциями sort/rsort (по значениям с переиндексацией), asort/arsort (с сохранением ключей), ksort/krsort (по ключам) и usort/uasort/uksort (пользовательский компаратор). Операции над множествами: array_merge (слияние), + (объединение с сохранением ключей первого массива), array_diff (разность), array_intersect (пересечение), array_unique (уникальные значения).

Стандартная библиотека PHP (SPL) добавляет готовые структуры данных с предсказуемой семантикой и лучшей асимптотикой: SplStack (стек LIFO), SplQueue (очередь FIFO), SplPriorityQueue (приоритетная очередь), SplHeap/SplMinHeap/SplMaxHeap (кучи), SplFixedArray (фиксированный по размеру массив), SplObjectStorage (набор объектов с метаданными). Для обхода и фильтрации данных предназначены итераторы: ArrayIterator, FilterIterator, CallbackFilterIterator, LimitIterator, AppendIterator, а также рекурсивные итераторы для вложенных структур.

Собственный итератор создаётся наследованием класса FilterIterator с переопределением метода accept(): bool – он вызывается для каждого элемента базового итератора, и элемент включается в итоговый обход, только если метод вернул true. Такой итератор можно использовать в конструкции foreach, что даёт ленивую фильтрацию без создания промежуточных массивов.

Генераторы (ключевое слово yield) позволяют итерироваться по «ленивому» потоку данных, не загружая всю коллекцию в память, что критично при обработке больших объёмов. Интерфейсы коллекционного поведения – Countable (count($obj)), ArrayAccess (доступ как к массиву), Iterator/IteratorAggregate (foreach), JsonSerializable (сериализация в JSON) – позволяют проектировать собственные коллекции, ведущие себя как встроенные.

Индивидуальное задание.

Вариант 10:

1. Задача (массивы): перевести плоский массив в двумерный, разбив на группы по M элементов;

2. Задача (SPL): реализовать собственный FilterIterator, оставляющий только строки длиной более 3 символов;

3. Задача (HTML/CSS/PHP): форма поиска с полем ввода и кнопкой. При вводе значения – подсветить найденный элемент в таблице.

Решение реализовано в виде веб-приложения. Пользователь вводит список строк (построчно или через запятую) и размер группы M; после отправки формы методом POST выводится двумерная таблица групп и список строк длиной более 3 символов. Поле поиска подсвечивает совпадающие элементы в обоих блоках результата. Все выводы экранируются через htmlspecialchars, что защищает от XSS.

Листинг кода.

index.php:

```php
<?php
declare(strict_types=1);

require_once __DIR__ . '/src/ArrayUtils.php';
require_once __DIR__ . '/src/LongStringFilter.php';
require_once __DIR__ . '/src/helpers.php';

use Lab2\ArrayUtils;
use Lab2\LongStringFilter;

$input_raw = (string)($_POST['items'] ?? '');
$group_size_raw = (string)($_POST['group_size'] ?? '3');
$search_query = trim((string)($_POST['search'] ?? ''));

$items = [];
$group_size = 3;
$grid = [];
$filtered = [];
$errors = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Валидация размера группы
    if (!is_numeric($group_size_raw) || (int)$group_size_raw < 1) {
        $errors[] = 'Размер группы M должен быть целым числом не меньше 1.';
        $group_size = 3;
    } else {
        $group_size = (int)$group_size_raw;
    }

    $items = parse_input($input_raw);

    if ($items === []) {
        $errors[] = 'Введите хотя бы одну строку (по одной на строку или через запятую).';
    }

    if ($errors === []) {
        // Задача 1 (массивы): плоский массив -> двумерный по группам по M элементов
        $grid = ArrayUtils::chunk_to_grid($items, $group_size);

        // Задача 2 (SPL): свой FilterIterator – строки длиной более 3 символов
        $iterator = new ArrayIterator($items);
        $filter = new LongStringFilter($iterator);
        foreach ($filter as $key => $value) {
            $filtered[$key] = $value;
        }
    }
}
?>
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
    <title>Лабораторная №2 – Вариант 10</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
<main class="container">
    <h1>Лабораторная работа №2 – Вариант 10</h1>
    <p class="subtitle">Разбиение массива на группы &middot; Spl FilterIterator &middot; Поиск
с подсветкой</p>

    <?php if ($errors !== []): ?>
        <div class="notice notice-error" role="alert">
            <strong>Ошибка ввода:</strong>
            <ul>
                <?php foreach ($errors as $err): ?>
                    <li><?= htmlspecialchars($err, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')
?></li>
                <?php endforeach; ?>
            </ul>
        </div>
    <?php endif; ?>

    <form method="post" class="input-form" novalidate>
        <label for="items">Список строк (по одной на строку или через запятую):</label>
        <textarea id="items" name="items" rows="7" required
                  placeholder="яблоко, груша, слива, ананас, мандарин"><?=
            htmlspecialchars($input_raw, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')
        ?></textarea>

        <div class="row">
            <div>
                <label for="group_size">Размер группы M:</label>
                <input type="number" id="group_size" name="group_size" min="1" max="50"
                       value="<?= htmlspecialchars($group_size_raw, ENT_QUOTES |
ENT_SUBSTITUTE, 'UTF-8') ?>"
                       required>
            </div>
            <div class="search-box">
                <label for="search">Поиск по таблице (задача 3):</label>
                <input type="text" id="search" name="search"
                       value="<?= htmlspecialchars($search_query, ENT_QUOTES | ENT_SUBSTITUTE,
'UTF-8') ?>"
                       placeholder="введите значение для подсветки…">
            </div>
        </div>

        <div class="actions">
            <button type="submit">Обработать</button>
            <button type="button" class="secondary"
                    onclick="document.getElementById('items').value='';
                             document.getElementById('search').value='';">Очистить</button>
        </div>
    </form>

    <?php if ($_SERVER['REQUEST_METHOD'] === 'POST' && $errors === []): ?>
        <section class="result">
            <h2>1. Двумерный массив (группы по <?= $group_size ?> элементов)</h2>
```

```
            <p>Элементов: <strong><?= count($items) ?></strong>,
                групп: <strong><?= count($grid) ?></strong></p>
            <table class="data-table">
                <thead>
                    <tr>
                        <th>Группа</th>
                        <?php for ($i = 0; $i < $group_size; $i++): ?>
                            <th>Элемент <?= $i + 1 ?></th>
                        <?php endfor; ?>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach ($grid as $row_index => $row): ?>
                        <tr>
                            <td class="group-num">№<?= $row_index + 1 ?></td>
                            <?php for ($i = 0; $i < $group_size; $i++): ?>
                                <?php
                                $cell = $row[$i] ?? '';
                                $is_match = $search_query !== ''
                                    && mb_stripos($cell, $search_query, 0, 'UTF-8') !== false;
                                ?>
                                <td class="<?= $is_match ? 'highlight' : '' ?>">
                                    <?= htmlspecialchars($cell, ENT_QUOTES | ENT_SUBSTITUTE,
'UTF-8') ?>
                                </td>
                            <?php endfor; ?>
                        </tr>
                    <?php endforeach; ?>
                </tbody>
            </table>

            <h2>2. Spl FilterIterator – строки длиной &gt; 3 символов</h2>
            <p>Исходных строк: <strong><?= count($items) ?></strong>,
                после фильтрации: <strong><?= count($filtered) ?></strong></p>
            <ul class="chip-list">
                <?php foreach ($filtered as $value): ?>
                    <li class="<?= ($search_query !== '' && mb_stripos($value, $search_query,
0, 'UTF-8') !== false) ? 'highlight' : '' ?>">
                        <?= htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8') ?>
                        <span class="len">(<?= mb_strlen($value, 'UTF-8') ?> симв.)</span>
                    </li>
                <?php endforeach; ?>
                <?php if ($filtered === []): ?>
                    <li class="empty">Нет строк длиной более 3 символов.</li>
                <?php endif; ?>
            </ul>
        </section>
    <?php endif; ?>

    <footer class="notes">
        <p>Задача 1 реализована вручную (без array_chunk). Задача 2 – собственный класс
            FilterIterator. Задача 3 – поиск с подсветкой совпадений через CSS-класс.</p>
    </footer>
</main>
</body>
</html>
```

src/ArrayUtils.php:

```php
<?php
declare(strict_types=1);

namespace Lab2;

/**
 * Утилиты для работы с массивами (вариант 10, задача 1).
 */
final class ArrayUtils
{
    /**
     * Переводит плоский массив в двумерный, разбивая его на группы
     * по $size элементов (реализация без array_chunk).
     *
     * Последняя группа может содержать меньше элементов.
     *
     * @param array<int,string> $items исходный плоский массив
     * @param int $size размер группы (M >= 1)
     * @return array<int,array<int,string>> двумерный массив
     * @throws \InvalidArgumentException если $size < 1
     */
    public static function chunk_to_grid(array $items, int $size): array
    {
        if ($size < 1) {
            throw new \InvalidArgumentException('Размер группы должен быть >= 1');
        }

        $grid = [];
        $group = [];

        foreach ($items as $item) {
            $group[] = $item;
            if (count($group) === $size) {
                $grid[] = $group;
                $group = [];
            }
        }

        // Неполная последняя группа
        if ($group !== []) {
            $grid[] = $group;
        }

        return $grid;
    }
}
```

src/LongStringFilter.php:

```php
<?php
declare(strict_types=1);
```

```php
namespace Lab2;

use FilterIterator;
use Iterator;

/**
 * Собственный итератор-фильтр (вариант 10, задача 2):
 * оставляет только строки длиной более 3 символов.
 */
final class LongStringFilter extends FilterIterator
{
    private const MIN_LENGTH = 3;

    /**
     * @param Iterator $iterator базовый итератор
     */
    public function __construct(Iterator $iterator)
    {
        parent::__construct($iterator);
    }

    /**
     * Принимает текущий элемент, если его длина больше 3 символов.
     */
    public function accept(): bool
    {
        $value = $this->current();

        if (!is_string($value)) {
            return false;
        }

        return mb_strlen($value, 'UTF-8') > self::MIN_LENGTH;
    }
}
```

src/helpers.php:

```php
<?php
declare(strict_types=1);

/**
 * Вспомогательные функции (не привязаны к пространству имён).
 */

/**
 * Разбирает пользовательский ввод в массив строк.
 *
 * Поддерживаются форматы:
 * - построчный ввод (каждая строка – элемент);
 * - ввод через запятые.
 *
```

```
     * @param string $raw сырой ввод из формы
     * @return array<int,string>
     */
    function parse_input(string $raw): array
    {
        $raw = str_replace(["\r\n", "\r"], "\n", $raw);

        $items = [];
        foreach (explode("\n", $raw) as $line) {
            $line = trim($line);
            if ($line === '') {
                continue;
            }

            if (mb_strpos($line, ',') !== false) {
                foreach (explode(',', $line) as $part) {
                    $part = trim($part);
                    if ($part !== '') {
                        $items[] = $part;
                    }
                }
            } else {
                $items[] = $line;
            }
        }

        return $items;
    }
```

css/style.css:

```
    :root{
      --bg:#f7f8fb;
      --card:#ffffff;
      --accent:#2b6cb0;
      --muted:#6b7280;
      --highlight:#fff3cd;
      --border:#e5e7eb;
      --error:#ffe8e8;
    }

    *{box-sizing:border-box}
    body{
      margin:0;
      font-family:Inter,"Segoe UI",Roboto,Arial,sans-serif;
      background:var(--bg);
      color:#111827;
      -webkit-font-smoothing:antialiased;
      padding:24px;
    }
    .container{
      max-width:960px;
      margin:0 auto;
      background:var(--card);
      padding:20px 24px;
```

```
  border-radius:8px;
  box-shadow:0 6px 18px rgba(15,23,42,0.06);
}
h1{margin-top:0;font-size:1.4rem;color:var(--accent)}
.subtitle{margin-top:-8px;color:var(--muted);font-size:0.95rem}
h2{font-size:1.1rem;color:var(--accent);margin-top:28px;
   border-bottom:2px solid var(--border);padding-bottom:6px}

.input-form label{display:block;margin-bottom:6px;font-weight:600}
textarea,input[type="text"],input[type="number"]{
  width:100%;
  padding:10px;
  border:1px solid var(--border);
  border-radius:6px;
  font-family:monospace;
  font-size:0.95rem;
}
textarea{min-height:110px}
.row{
  display:flex;
  gap:16px;
  margin-top:12px;
}
.row>div{flex:1;min-width:0}
input[type="number"]{max-width:160px}

.options{margin:10px 0}
.actions{display:flex;gap:10px;margin-top:14px}
button{
  background:var(--accent);
  color:#fff;
  border:0;
  padding:9px 16px;
  border-radius:6px;
  cursor:pointer;
  font-size:0.95rem;
}
button.secondary{background:#6b7280}
button:hover{filter:brightness(1.08)}

.result{margin-top:10px}
.data-table{
  width:100%;
  border-collapse:collapse;
  margin-top:12px;
  font-size:0.95rem;
}
.data-table th,.data-table td{
  border:1px solid var(--border);
  padding:8px 10px;
  text-align:left;
}
.data-table thead th{background:#f3f4f6;font-weight:600}
.data-table td.group-num{font-weight:600;color:var(--muted);white-space:nowrap}
.data-table td:empty::after{content:"–";color:var(--muted)}
```

```
/* Подсветка найденного элемента (задача 3) */
.highlight{
  background:linear-gradient(90deg,rgba(255,243,205,0.9),rgba(255,243,205,0.45));
  outline:2px solid #ecc94b;
}

.chip-list{list-style:none;padding:0;margin:12px 0 0;display:flex;flex-wrap:wrap;gap:8px}
.chip-list li{
  padding:6px 12px;
  border:1px solid var(--border);
  border-radius:999px;
  background:#f9fafb;
  font-size:0.95rem;
}
.chip-list li .len{color:var(--muted);font-size:0.8rem}
.chip-list li.empty{border-style:dashed;color:var(--muted)}

.notice{
  padding:12px 14px;
  border-radius:6px;
  margin:14px 0;
}
.notice-error{background:var(--error);border:1px solid #fca5a5}
.notice ul{margin:6px 0 0;padding-left:20px}

.notes{margin-top:26px;color:var(--muted);font-size:0.85rem;
       border-top:1px solid var(--border);padding-top:10px}

@media (max-width:600px){
  body{padding:12px}
  .row{flex-direction:column;gap:10px}
  input[type="number"]{max-width:100%}
  .data-table{font-size:0.85rem}
  .data-table th,.data-table td{padding:6px}
}
```

Пояснение к решению.

1. В задаче 1 функция ArrayUtils::chunk_to_grid() выполняет линейный проход по одномерному массиву, последовательно формируя строки двумерной матрицы фиксированного размера M. Элементы накапливаются во временный буфер; при достижении размера M строка фиксируется в результирующей матрице. Оставшиеся элементы последней неполной строки добавляются после завершения цикла. Для недопустимого значения M < 1 генерируется исключение InvalidArgumentException.

2. В задаче 2 разработан класс LongStringFilter, расширяющий стандартный FilterIterator библиотеки SPL. В переопределённом методе accept() выполняется валидация типа и вычисление длины строки с помощью многобайтовой функции mb_strlen(..., 'UTF-8'), что гарантирует корректную

фильтрацию строк длиной строго более 3 символов в кодировке UTF-8. Фильтрация выполняется в ленивом потоковом режиме, без выделения дополнительной памяти под промежуточные массивы.

3. В задаче 3 поиск по сформированной таблице реализован с использованием функции mb_stripos (регистронезависимое совпадение). Ячейки, удовлетворяющие условию поиска, динамически помечаются CSS-классом highlight с визуальным цветовым акцентом и рамкой.

Результаты работы.

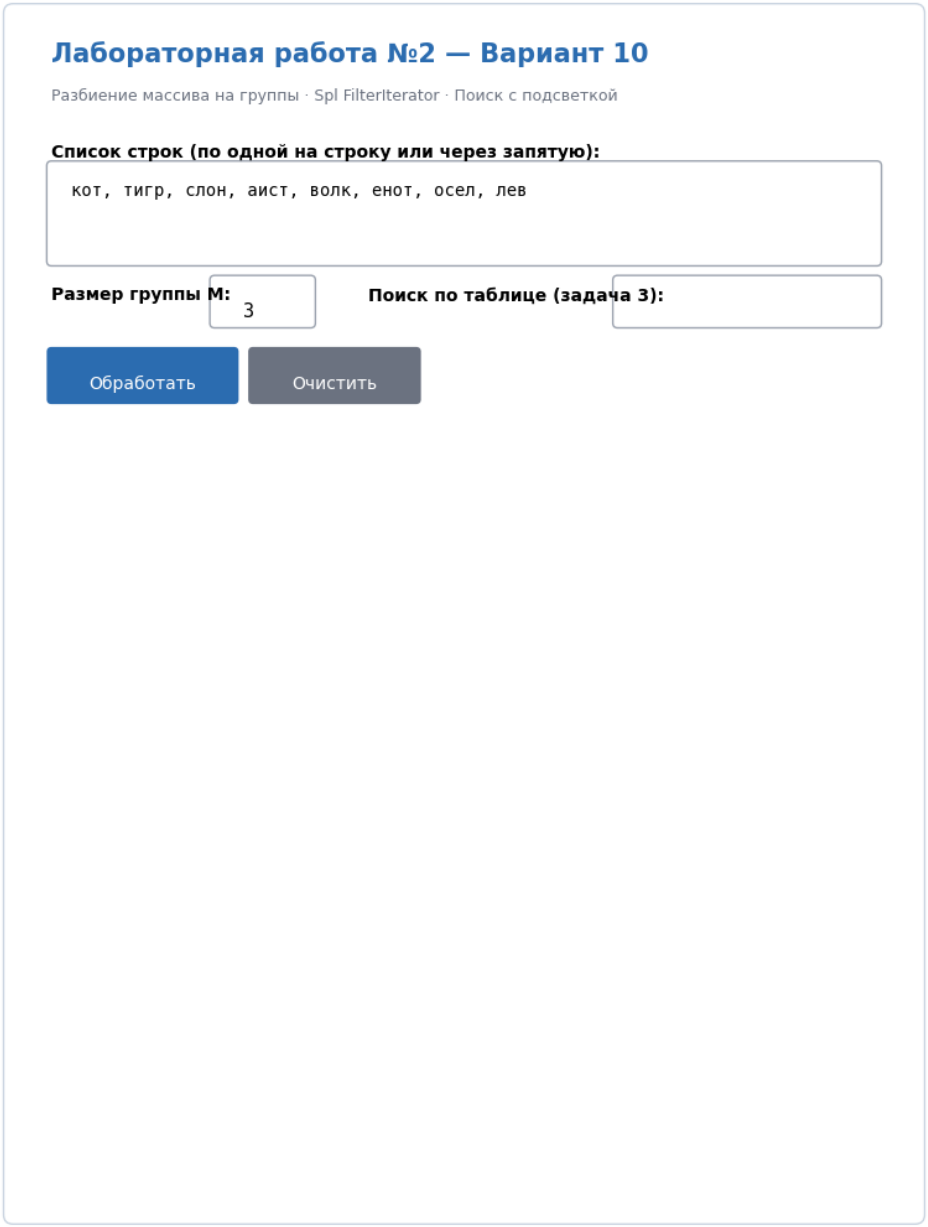


Рисунок 1 – Страница ввода данных (форма)

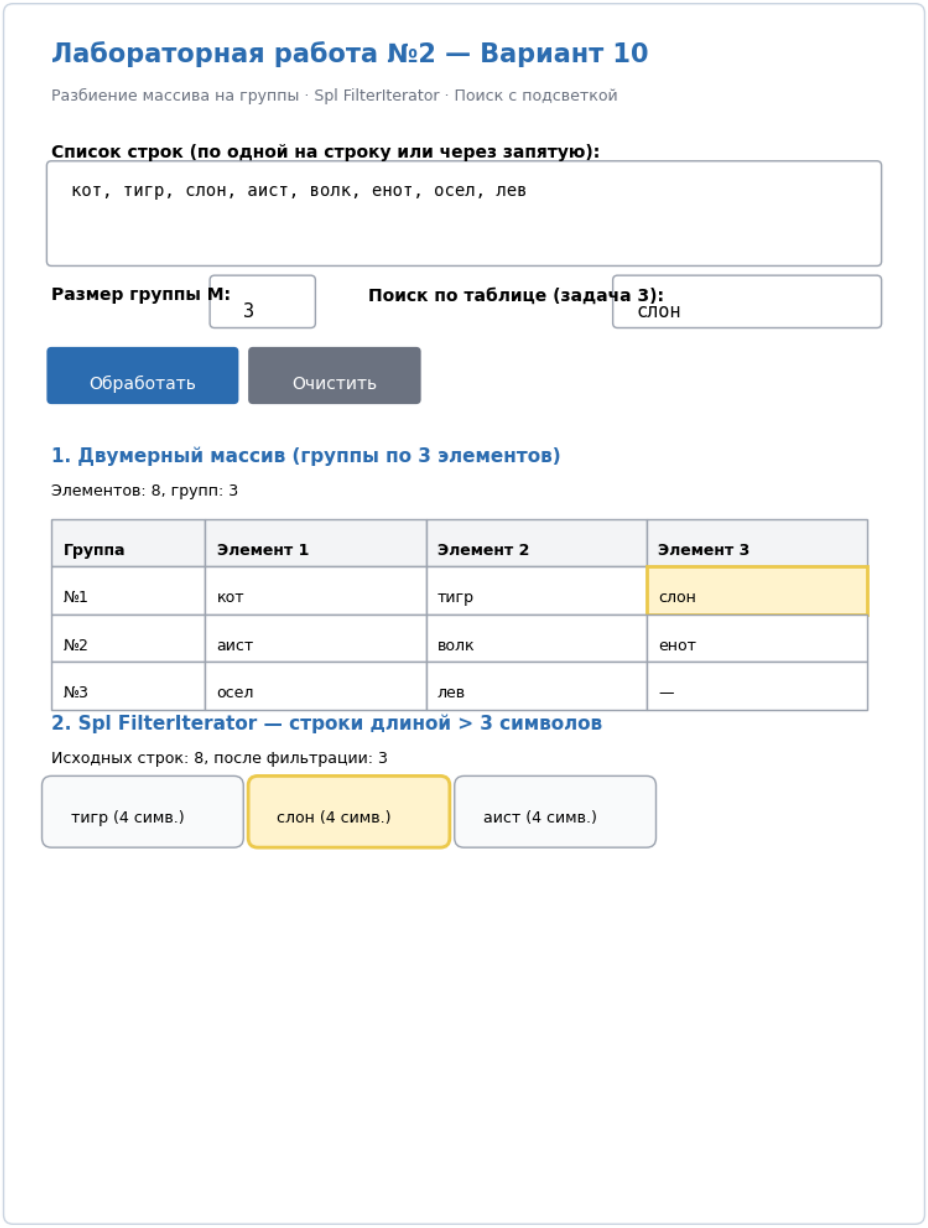


Рисунок 2 – Результат обработки с подсветкой найденного элемента

Выводы.

В ходе лабораторной работы были изучены ключевые коллекции и структуры данных языка PHP: индексные, ассоциативные и многомерные массивы, функции функциональной обработки (array_map, array_filter, array_reduce), а также специализированные итераторы и структуры стандартной библиотеки SPL. На практике были успешно решены три задачи: формирование двумерной сетки из плоского массива по заданному размеру группы без использования вспомогательной функции array_chunk, реализация собственного

потокового фильтра на базе FilterIterator для отбора строк длиной более 3 символов и разработка адаптивного веб-интерфейса с формой поиска и подсветкой результатов. Освоены приёмы строгого типизирования, документации phpDoc, безопасного экранирования вывода и адаптивной вёрстки.